

LLM-IAQ: 基于权重重要性的统一精度量化方法

SiCheng Feng¹ Yipeng Du¹

¹ Nankai University

<https://github.com/natsunoshion/llm-iaq>

摘要

sunoshion@gmail.com

大语言模型 (LLMs) 在多种自然语言处理任务 (e.g., 机器翻译, 情感分析, 阅读理解, 对话系统, 问答系统) 中取得了显著进展。在实际应用中, 尤其是在资源有限的情况下 (如边缘设备等), 其庞大的模型规模和高昂的计算成本带来了挑战。本地运行 LLM 不仅能降低云计算成本, 还能保护用户隐私。为了解决上述问题, 量化 (Quantization) 是减少模型内存占用和加速推理的重要方法, 但传统量化方法可能因量化误差导致性能下降, 限制了大模型的直接部署。近年来, 针对 LLM 的量化方法取得了显著进展, 尤其是在保护模型性能的同时显著降低硬件资源需求方面。本文对于 LLM 量化进行了分析, 并对现有方法的挑战与未来研究方向进行探讨, 提出了一种基于权重重要性分级放大的统一精度量化方法: 该方法通过结合权重的 L1 范数分析和多级动态缩放策略, 在保持模型性能的同时实现高效的硬件推理支持。在之后的期末项目里, 我们会参照此 Proposal, 立志于完成一项扎实的工作。

1. 小组成员信息

1. Sicheng Feng (冯思程)¹: 学号 2112213, 邮箱 fscnkucs@gmail.com
2. Yipeng Du (杜旖苒)²: 学号 2113958, 邮箱 nat-

¹复现部分: AWQ 和 Smooothquant; 创新部分 idea 提出和实现, 以及小部分实验运行与结果分析。

²复现部分: LLM.int8(); 创新部分协助实现, 以及大部分实验完成以及结果分析。

2. 引言

大语言模型 (LLMs) 在多种自然语言处理任务中取得了突破性进展, 例如机器翻译 [34, 11]、情感分析 [8]、阅读理解、对话系统 [33] 和问答系统 [2], 基本架构是 Transformer [27]。然而, LLMs 的出色表现背后是庞大的参数量和计算需求, 这带来了实际应用中的种种挑战, 尤其在资源受限的环境下 (如边缘设备等)。例如, 许多先进的 LLMs 模型需要数百 GB 的存储空间和大量的 GPU 计算资源 (e.g., GPT-175B [2] 拥有 1750 亿个参数, 在半精度 (FP16) 格式下至少需要 350GB 的内存, 并且推理需要至少五张 80GB 内存的 A100 GPU。), 这使得它们在部署时面临巨大的硬件成本和能耗问题。因此, 如何在不显著降低模型性能的情况下降低其计算和存储开销成为了亟待解决的课题。

为应对这些挑战, 模型压缩技术 [13] 逐渐成为优化 LLMs 的关键手段。模型压缩通过将大型、高资源需求的模型转化为适合资源有限环境的紧凑版本, 从而提升模型的推理速度并减少资源消耗。本文从四个核心技术角度探讨了 LLMs 的模型压缩, 包括量化、剪枝、知识蒸馏和低秩分解。量化技术通过将模型参数表示为更低精度的数据类型来减少模型大小和计算需求 [17]; 剪枝方法则通过去除冗余参数来简化模型结构 [20]; 知识蒸馏则是利用一个预训练的大模型 (教师模型) 来指导较小的模型 (学生模型) 学习, 从而在小模型中保留原始大模型的部分能力 [14]; 而低秩分解方法则将权重矩阵分解为

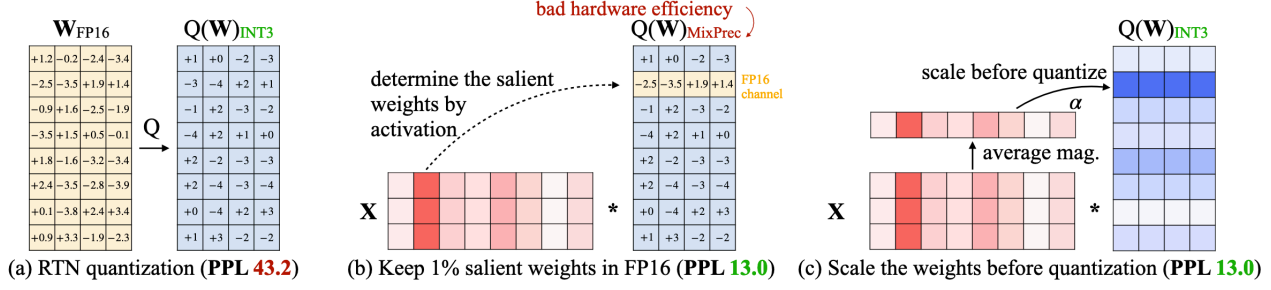


图 1. AWQ [17] 示意图。根据激活分布（中间），我们可以在 LLM 中找到 1% 的突出权重。将突出权重保留在 FP16 中可以显著提高量化性能（PPL 从 43.2（左图）降至 13.0（中图）），但混合精度格式不具备硬件效率。AWQ（右图）对每个通道进行缩放，以保护突出权重并减少量化误差。

更小的矩阵，进一步降低了计算复杂度 [24]。这些技术的相结合在一定程度上缓解了 LLMs 在实际应用中的部署难题。

本项目在以量化为主要研究方向，研究量化技术，旨在能够通用性地减少大型语言模型对计算资源的需求，让大模型有机会在资源限制的系统上运行。

3. 问题定义

大模型压缩指的是将大型资源密集型模型转化为更紧凑的版本，以便在资源有限的设备上部署，从而提高推理速度并优化资源效率。这在 NLP 任务里是一个前沿且具有潜力的方向，特别是当前许多大模型如 GPT、LLaVA 等，需要占据很大显存才能运行；而经过压缩后的大模型则有望部署于小型移动设备。压缩后模型的性能可以通过推理时间、压缩比、加速比等指标来评估，进而实现大语言模型的高效部署。常见的指标包括模型大小、推理时间、准确率、FLOPs 等。

我们分析仅权重量化的误差。考虑权重 w ；线性运算可写为 $y = wx$ ，量化对应项为 $y = Q(w)x$ 。具体来说，量化函数可定义为：

$$Q(w) = \Delta \cdot \text{Round}\left(\frac{w}{\Delta}\right), \quad \Delta = \frac{\max(|w|)}{2^{N-1}}, \quad (1)$$

其中， N 是量化位数， Δ 是由绝对最大值确定的量化缩放因子。

4. 相关工作

大型语言模型 (LLM) 近年来收获众多关注。大模型量化 [12] 是一种减少模型参数位数（即精度）的技术，以降低存储和计算成本，同时尽可能保持模型性能。量化可分为量化感知训练 (QAT) 和训练后量化 (PTQ)。QAT 需要对模型进行再训练以校正因量化带来的误差，而 PTQ 则直接在推理阶段应用量化模型。

4.1. 量化感知训练 (QAT)

QAT 通过再训练提升模型性能，减少量化带来的精度损失。例如，LLM-QAT [18] 使用自我蒸馏生成数据进行训练，BitDistiller [9] 结合自蒸馏，优化至低于 4 位的量化效果，而 OneBit 提出 1 位参数量化方法，实现极低位数的量化部署。QAT 可以有效减少精度损失，但由于 LLM 参数量巨大，再训练开销高。为此，结合参数高效微调 (PEFT) 的方法，如 QLORA 等，可以提高 QAT 效率，但这些方法多依赖特定任务。

4.2. 训练后量化 (PTQ)

PTQ 在不需要再训练的情况下，将全精度模型转换为低精度版本。PTQ 主要分为**权重量化**、**权重-激活量化**和 **KV 缓存量化**。权重量化仅对权重进行量化，而权重-激活量化则扩展至激活值。研究表明，激活量化对权重量化更敏感，因此权重量化可以在更低的位宽下实现。KV 缓存量化则针对 KV 缓存（用于存储注意力层的键和值）进行量化，减少内存占用并提升长序列处理的效率。

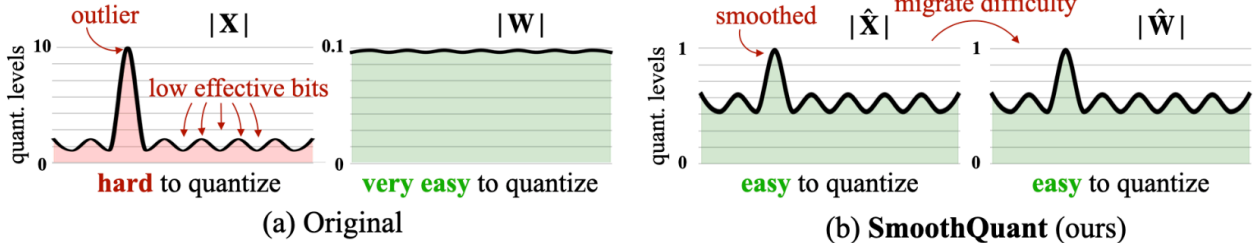


图 2. SmoothQuant [28] 的直觉是：激活 X 很难量化，因为异常值会拉大量化范围，使大部分值的有效比特数很少。我们在离线过程中将比例方差从激活值迁移到权重 W ，以降低激活值的量化难度。平滑后的激活值 $\hat{X}$ 和调整后的权重 $\hat{W}$ 都很容易量化。

权重量化. 权重量化是一种传统且广泛应用的方法。例如，LUT-GEMM [23] 使用二进制编码实现矩阵乘法加速，GPTQ [10] 采用基于逆 Hessian 信息的层级量化方法，QuIP [3] 利用 Hessian 矩阵的 LDL 分解进行 2 位量化。为了进一步减少量化误差，一些方法会保留敏感权重的高精度表示，例如 AWQ [17] 存储最重要的 1% 权重，以优化量化效果。

权重-激活量化. 许多研究致力于权重-激活量化，其中 ZeroQuant [29] 首先实现了 LLM 的权重-激活量化。为处理激活值中的异常值，LLM.int8() [6] 专门保留了这些异常值的高精度。SmoothQuant [28] 设计了通道级的平滑变换，减少了激活值中的异常波动。其他方法如 RPTQ [30] 也采用了各种方式处理通道间的数值差异或异常值，以改善量化效果。

KV 缓存量化. 随着 LLM 输入长度的增加，KV 缓存的内存需求也随之增长。KVQuant [15] 等方法提出了通道级和非均匀量化技术，以提升长序列推理的效率。KIVI [19] 则通过分析 KV 缓存中的元素分布，发现可以分别对键缓存和值缓存采用不同的量化策略，成功将 KV 缓存量化到 2 位。WKVQuant [31] 采用二维量化策略管理 KV 缓存，实现了接近权重-激活量化的内存节省效果。

5. 方法

5.1. 复现论文方法描述

AWQ [17]. LLMs 的设备端部署 (on-device LLM) 在降低云计算成本和保护用户隐私方面具有重要意

义。然而，模型规模庞大和硬件资源有限性带来了巨大挑战。论文提出了一种硬件友好的低比特权重量化方法——激活感知权重量化 (Activation-aware Weight Quantization, AWQ)。AWQ 发现，仅保护 1% 的关键权重即可显著降低量化误差，这些权重的选择需参考激活分布而非权重本身。如图 1 所示。

为了避免混合精度量化的硬件低效问题，AWQ 通过数学推导证明，放大关键权重通道的比例可以减少量化误差，并通过等效变换保护关键通道，比例因子则基于离线激活统计确定。无需反向传播或重构，AWQ 能够很好地泛化到不同领域和模态，避免过拟合。

AWQ 在语言建模、指令调优和多模态任务上均表现优异，是首次在多模态模型上取得如此出色的量化效果。同时，论文还实现了高效的推理框架 TinyChat，通过内核融合和权重优化，在桌面和移动 GPU 上比 Huggingface FP16 实现提速 3 倍以上，并成功将 70B Llama-2 模型部署至移动 GPU。

SmoothQuant [28]. LLMs 尽管性能优异，但对计算和内存资源要求极高。量化可以减少内存使用并加速推理，但现有方法难以同时兼顾精度和硬件效率。论文提出了 SmoothQuant，如图 2 所示，一种无需训练、精度友好且通用的 PTQ 方法，可实现 LLM 的 8 比特权重和 8 比特激活 (W8A8) 量化。

基于权重易于量化而激活较难的特点，SmoothQuant 通过数学等效变换，离线地将量化难度从激活迁移至权重，从而平滑激活的离群值。这一方法支持所有矩阵乘法运算的 INT8 量化，适用于

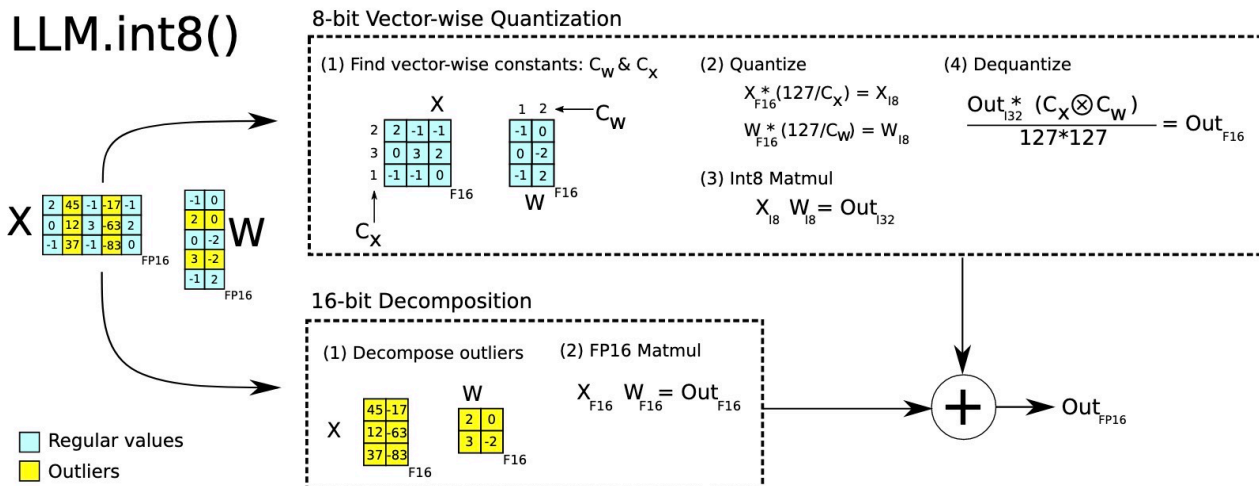


图 3. LLM.int8() [6] 的示意图。在给定 16 位浮点输入 X_{f16} 和权重 W_{f16} 的情况下，特征和权重被分解成大振幅特征子矩阵和其他值。离群特征矩阵以 16 位乘法运算。所有其他值以 8 位乘法运算。我们通过 C_x 和 C_w 的绝对最大值按行和列进行缩放，执行 8 位矢量乘法，然后将输出量化为 Int8。Int32 矩阵乘法输出 Out_{i32} 通过归一化常数 $C_x C_w$ 的外积进行去量化。最后，离群值和正则输出都被累加为 16 位浮点输出。

包括 OPT、BLOOM、GLM、MT-NLG、Llama-1/2、Falcon、Mistral 和 Mixtral 在内的多种模型。

实验表明，SmoothQuant 在几乎无损精度的情况下，实现了最高 1.56 倍的推理速度提升和 2 倍的内存占用减少，使得单节点上服务 530B 参数模型成为可能。该方法为 LLM 的硬件成本优化提供了开箱即用的解决方案，加速了 LLM 的普及与应用。

LLM.int8() [6]. LLMs 被广泛应用，但推理过程对 GPU 内存需求极大。论文提出了一种用于 Transformer 模型中前馈层和注意力投影层的 Int8 矩阵乘法方法，可以将推理所需内存减半，同时保持全精度性能。

通过该方法，可直接加载 175B 参数的 16/32 位模型检查点，转换为 Int8 格式后立即使用，且无性能损失。这得益于对 Transformer 语言模型中高度系统化的显著特征的理解和处理，这些特征主导了注意力机制和预测性能。为应对这些特征，论文开发了一种两阶段量化方法——LLM.int8()，如图 3 所示。第一阶段使用向量级量化，为矩阵乘法中的每个内积分配独立的归一化常数，以量化大多数特征；第二阶段对突出的离群特征维度采用混合精度分解方案，将其隔离至 16 位矩阵乘法中，而 99.9% 以上

的值仍通过 8 位运算处理。

实验证明，使用 LLM.int8() 可以在不损失性能的情况下推理最多 175B 参数的 LLM。这一成果显著提升了模型的可访问性，例如支持在单台消费级 GPU 服务器上运行 OPT-175B 或 BLOOM 模型。

5.2. 复现难点以及细节

环境配置。在环境配置中由于量化技术比较偏向底层，其中涉及 CUDA 编程，所以对环境的要求比较苛刻，比如 Smoothquant 要求 CUDA 版本是 11.3；AWQ 要求 CUDA 版本需要是 12.x。这里我主要用到了 Docker 来进行环境管理，我们分别用了一个 CUDA11.3 和 CUDA12.2 的 image 来启动了我们所需环境的容器，并进行了路径映射。

推理框架。AWQ 结合了推理框架 TinyChat 以提升运行效率，我们关注框架中内核融合、计算图优化等细节，特别是 AWQ 的内核代码部分，基本都是 C++ 和 CUDA 编写，阅读难度较大。此外，AWQ 强调保护激活分布中 1% 的关键权重，这些权重的选择依赖于离线统计激活分布。具体实现中，需对每一层计算激活分布并根据论文提供的数学公式确定关键通道的策略。

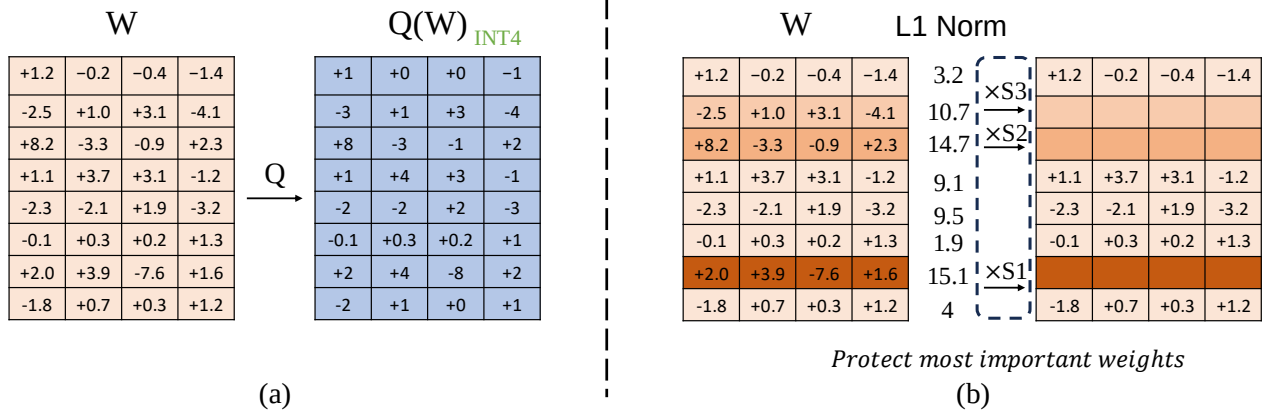


图 4. IAQ 示意图。(a) 表示直接进行 INT 量化方法，往往无法保留重要信息，而我们的方法 (b) 能够根据权重 W 的 L1 范数来对前 1% 的值进行层级地优化，乘上不同的权重因子 S 体现不同的保护。

其他细节。各量化方法的优势在于通用性强，能够泛化到多种任务和模型，在复现实验中，我们要测试不同规模和架构（如 Llama、OPT）模型的表现。

5.3. 提出的创新方法 - IAQ

这里我们尝试提出一种**基于权重重要性的统一精度量化方法**。该方法通过结合权重的 L1 范数分析和多级动态缩放策略，在保持模型性能的同时实现高效的硬件推理支持。该思路主要来源于前文的 AWQ 方法和剪枝领域的 L_1 -norm pruning [16]。主要创新点如下：

首先，我们对模型的权重进行重要性分析，采用 L1 范数作为衡量标准，将权重分为不同重要性梯度。重要性高的权重通常对模型性能贡献较大，因此需要更高的保护。而低重要性的权重可以使用较低精度表示，从而达到压缩模型的目的。其次，为了解决硬件对统一量化精度的需求，我们对高重要性权重采用动态缩放策略，将其数值范围放大到更适合低比特量化的范围。通过这种数学等效变换，可以减少量化误差的影响，同时保证计算的一致性。在推理阶段，我们通过对激活值进行分组反缩放，确保量化后的模型与原始模型在数学上等价，既简化了实现复杂度，也保证了模型的推理效率。

技术路线分为三个关键步骤：

1. 计算权重的 L1 范数并进行重要性排序，根据预

设的分位点将权重划分为高、中、低三个梯度（或者更多梯度，甚至可以尝试自适应选取梯度数，我们目前视线是自定义高中低三个梯度，未来尝试继续扩展）。

2. 对不同重要性梯度的权重分别应用不同的缩放因子，放大高重要性权重的数值范围以适配低比特量化（保护更加重要的权重，不同重要程序的权重保护的程序也不同，两者正相关），并对权重矩阵进行统一的低比特量化表示。
3. 在推理过程中，动态调整输入激活值以匹配权重的缩放因子，从而实现与原始模型的数学一致性（也就是权重放大了，输入对应的位置就要缩小，保证一致性）。

此方法的优势在于，通过对权重重要性分级和动态缩放的结合，能够有效保护关键权重，显著降低量化误差，同时以统一的低比特精度表示所有权重，兼顾模型压缩效率和硬件友好性。此外，整个流程无需重新训练模型，仅需小量的校准数据即可完成，从而具备良好的通用性和实际应用价值。

5.4. 创新方法实现细节

Kernel 构建 & 高效矩阵计算的 CUDA 实现。 Attention 部分使用 FasterTransformer。此外，我们参考 AWQ [17] 实现了 LayerNorm 和 Positional Embedding。

表 1. OPT-175B 模型下的性能表现，前 8 个任务为 0-shot 的准确率，最后为 Wiki 任务上的困惑度 PPL 得分。

OPT-175B	LAMBADA	HellaSwag	PIQA	WinoGrande	OpenBookQA	RTE	COPA	Average↑	WikiText↓
FP16	74.7%	59.3%	79.7%	72.6%	34.0%	59.9%	88.0%	66.9%	10.99
LLM.int8()	74.7%	59.2%	79.7%	72.1%	34.2%	60.3%	87.0%	66.7%	11.10
SmoothQuant-O1	74.7%	59.2%	79.7%	71.2%	33.4%	58.1%	89.0%	66.5%	11.11
SmoothQuant-O2	75.0%	59.0%	79.2%	71.2%	33.0%	59.6%	88.0%	66.4%	11.14
SmoothQuant-O3	74.6%	58.9%	79.7%	71.2%	33.4%	59.9%	90.0%	66.8%	11.17
AWQ	75.0%	60.1%	80.3%	72.3%	36.1%	59.5%	91.0%	69.3%	10.38
IAQ (ours)	74.9%	59.1%	79.9%	72.0%	34.9%	60.1%	90.7%	67.4%	10.37

$$\text{LayerNorm}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \cdot \gamma + \beta \quad (1)$$

$$\begin{aligned} PE_{(pos, 2i)} &= \sin\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right) \\ PE_{(pos, 2i+1)} &= \cos\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right) \end{aligned} \quad (2)$$

其中 pos 是位置索引， i 是维度索引， d_{model} 是模型的维度。

此外，我们参考 AWQ 实现了一个 4 位量化矩阵乘法（GEMM）操作，以加速深度学习模型中的前向计算。矩阵乘法的核心操作由以下公式给出：

$$Y = A \cdot B + C \quad (3)$$

其中， A 为输入矩阵（量化后的特征矩阵）， B 为量化后的权重矩阵， C 为输出矩阵，表示最终的前向计算结果。其中输入特征矩阵 A 和权重矩阵 B 被加载到共享内存中，以减少全局内存访问延迟。权重矩阵 B 被量化为 4 位，从而减少了内存的使用。在共享内存中对数据进行对齐，以优化访问模式，从而确保高效的计算。

矩阵乘法通过 CUDA 硬件加速的 `mma.sync` 指令进行执行。这一操作被设计为执行高效的矩阵乘法，减少了计算周期。对于量化后的矩阵，计算遵循以下公式：

$$B_{\text{quantized}} = \text{Scale}(B) \times (B - Z) \quad (4)$$

其中， B 为原始权重矩阵， Z 为零点， $\text{Scale}(B)$ 是通过量化过程确定的缩放因子。量化后，权重矩

阵 B 被存储为 4 位整数。在计算过程中，这些量化值被反量化为 16 位浮点值，进行后续的计算：

$$B_{\text{dequantized}} = \text{Scale}(B) \times (B_{\text{raw}} - Z) \quad (5)$$

其中， B_{raw} 表示量化后的权重值。

量化部分。对于权重量化，我们参考 AWQ 分别实现了伪量化和真实量化两个阶段。伪量化旨在模拟量化过程，主要通过以下步骤实现：

- **缩放因子计算：**通过以下公式计算权重张量的缩放因子：

$$\text{scale} = \frac{\text{max_val} - \text{min_val}}{2^n - 1}$$

其中， max_val 和 min_val 分别为权重的最大值和最小值， n 为量化位宽。

- **量化映射：**利用缩放因子将权重值从连续的浮点数空间映射到离散的整数空间：

$$q = \text{round}\left(\frac{x}{\text{scale}} + \text{zero_point}\right)$$

其中， zero_point 表示零点偏移，用于对称量化。

- **反量化恢复：**通过以下公式恢复到浮点数值：

$$x' = \text{scale} \cdot (q - \text{zero_point})$$

伪量化通过模拟量化操作，保留原始的 Linear 层，从而便于在训练或验证阶段验证量化效果。

真实量化直接将 Linear 层替换为支持量化的 WQLinear 层，用于实际部署。其过程包括：

- 使用伪量化得到的缩放因子和零点，初始化 WQLinear 层。

表 2. Llama 模型上, 各种量化方法在 WikiText-2-test 上的 PPL 得分和八个 0-shot 常识推理任务上的平均准确率。SQ 代表 Smooth Quant, INT 代表量化位数。↑: 越高越好, ↓: 越低越好。

Bit-width	Format	Method	Llama2-7B		Llama2-13B		Llama3-1.8B	
			0-shot↑	Wiki↓	0-shot↑	Wiki↓	0-shot↑	Wiki↓
A:16, W:16	A:FP16, W:FP16	N/A	59.89	5.12	62.80	4.57	63.60	5.61
A:8, W:8	A:INT8, W:INT8	SQ	58.69	5.15	61.61	4.60	62.60	5.63
		AWQ	58.23	5.17	61.58	4.62	62.74	5.62
		IAQ (ours)	58.20	5.21	61.62	4.62	62.71	5.62
A:8, W:4	A:INT8, W:INT4	SQ	55.27	6.34	57.34	5.46	54.51	6.11
		AWQ	57.64	5.61	60.86	4.85	61.89	6.94
		IAQ (ours)	57.20	5.71	60.69	4.91	61.70	7.03
A:8, W:3	A:INT8, W:INT3	SQ	-	-	32.40	35.24	32.20	35.24
		AWQ	37.15	8.12	50.00	8.43	39.34	12.94
		IAQ (ours)	36.23	9.17	51.58	9.62	38.74	13.12

- 将权重张量的存储和计算严格限制在离散的量化整数范围内。

真实量化大幅减少了存储和计算的开销, 同时适合资源受限的设备部署。

激活值的缩放通过插入 ScaledActivation 层实现, 用于调整激活值的动态范围。具体实现如下:

- 在 BloomBlock, MPTBlock, Falcon 等模块中, 为每个激活函数 (如 GELU) 添加对应的 ScaledActivation 层。
- 初始化激活值的缩放因子为全 1 (即未缩放), 后续根据量化需求动态调整。
- 动态调整激活值的缩放因子以减少量化误差。

6. 实验

6.1. 数据集

本次实验使用数据集包括 0-shot、WikiText-2 任务, 在 OPT 模型和 Llama 系列模型上测试量化性能与效果。我们在 WikiText-2 [21] 上测量了量化语言模型的困惑度 (Perplexity, 简称 PPL), 困惑度可靠地反映了 LLM 的性能 [7, 17]。此外, 为了评估量化模型在下游任务上的性能, 我们在八个零样本常

识推理任务上测量了它们的准确性, 包括 ARC-easy、ARC-challenge [5]、BoolQ [4]、PIQA [1]、SIQA [26]、HellaSwag [32]、OBQA [22] 和 WinoGrande [25] 任务。

6.2. 实验设置

小组成员分别有服务器可以访问并可以用于本项目, 本次实验在 8 卡 RTX 4090 以及 V100 上完成。

6.3. 结果与分析

在表 1 中, 我们的 IAQ 展现出了与 AWQ 可比的性能, 这体现在某些数据集上具有与 AWQ 等先进方法相当的准确率, 较为不错, 这证实了我们方法的有效性。尽管在部分数据集上有一些不足, 但能够超过 Smooth Quant 方法也能够说明我们的方法具有一定的开创性, 有其创新的价值。

从表 2 中可以看出, AWQ 量化方法在 0-shot 数据集以及 WikiText-2 数据集上取得了最优的性能, 其次我们的 IAQ 方法在拥有创新性的基础上完成了一次不错的实验, 依旧获得了不错的性能 (相对于 AWQ 较弱一点点, 但依旧能够超过 Smoot Quant

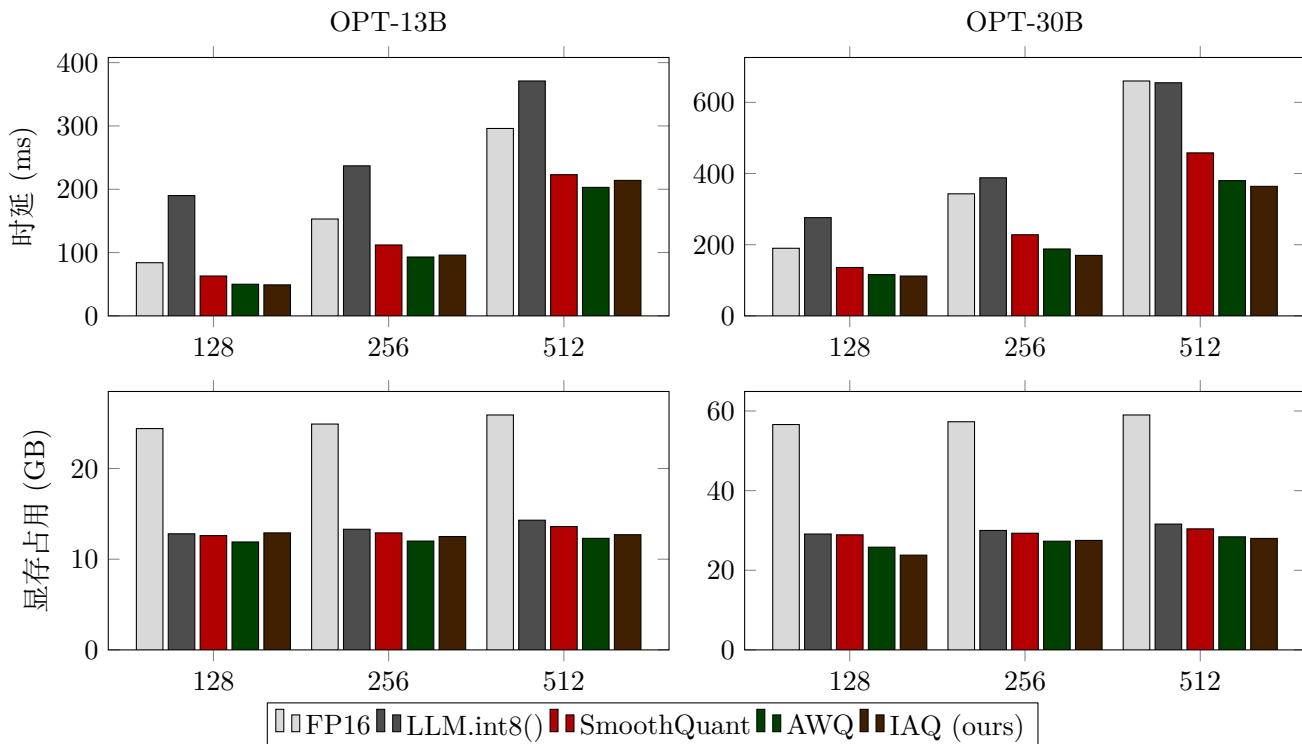


图 5. FP16, LLM.int8(), SmoothQuant, 以及我们的 IAQ 量化方法在 OPT-13B and OPT-30B 上的性能对比。

方法)。分析原因，我们的 L1 Norm 方法取得了不错的性能，但由于权重缩放因子的选择问题，可能不具备 AWQ 的自适应选择能力，在 Llama 模型上并未选择出最好的超参数，未来有进一步改进的空间。

从图 5 中可以看出，我们的 IAQ 方法依旧能够与 AWQ、Smooth Quant 等方法可比，同时在 OPT-30B 的模型上有超过 AWQ 的趋势，我们认为这很可能是因为我们的层次权重方法具有高效的数据利用性，能够保护前 1% 的重要权重值，因此在大模型中具备不小的潜力。

7. 限制与未来展望

本次实验从实验结果上看，可以发现依旧有改进空间，未来，我们认为可以考虑采用更加自适应的梯度等级个数，以及缩放因子自适应调整，这样我们的方法能够更好的性能。另一方面，激活值的长尾分布会导致异常值对模型性能的敏感性增大（即“离群值问题”）。尽管 SmoothQuant 等方法缓解了这一问题，但通用性和精度仍需提升，未来我们可

以探索离群值敏感度自适应的权重/激活联合量化方法。

8. 总结与展望

本项目围绕大语言模型（LLMs）的量化问题，提出了一种基于权重重要性分级放大的统一精度量化方法，旨在解决当前模型量化过程中性能损失和硬件效率的平衡问题。通过结合权重的 L1 范数分析和多级动态缩放策略，该方法在降低模型存储和计算成本的同时，保持了模型的性能，适配了硬件的统一精度需求。

本方法的潜在优势在于，不仅实现了统一的低比特精度量化，降低了硬件部署的复杂性，还无需重新训练，仅需少量校准数据即可完成量化处理，具备高效性和广泛的适用性。未来，我们计划在多个主流语言模型（如 GPT、Llama、OPT）上验证方法的有效性，并探索更多的实际应用场景（如边缘设备部署）。

本项目尝试为高效的大语言模型量化提供一种

新思路，为资源受限环境下的大模型推理提供技术支持。同时，我们希望通过本项目的研究，为进一步优化大语言模型的部署与应用打下坚实基础。

参考文献

- [1] Y. Bisk, R. Zellers, J. Gao, Y. Choi, et al. Piqa: Reasoning about physical commonsense in natural language. In Proceedings of the AAAI conference on artificial intelligence, volume 34, pages 7432–7439, 2020. [7](#)
- [2] T. B. Brown. Language models are few-shot learners. arXiv preprint arXiv:2005.14165, 2020. [1](#)
- [3] J. Chee, Y. Cai, V. Kuleshov, and C. M. De Sa. Quip: 2-bit quantization of large language models with guarantees. In NeurIPS, 2024. [3](#)
- [4] C. Clark, K. Lee, M.-W. Chang, T. Kwiatkowski, M. Collins, and K. Toutanova. Boolq: Exploring the surprising difficulty of natural yes/no questions. arXiv preprint arXiv:1905.10044, 2019. [7](#)
- [5] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. arXiv preprint arXiv:1803.05457, 2018. [7](#)
- [6] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer. Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale. In NeurIPS, 2022. [3, 4](#)
- [7] T. Dettmers and L. Zettlemoyer. The case for 4-bit precision: k-bit inference scaling laws. In International Conference on Machine Learning, pages 7750–7774. PMLR, 2023. [7](#)
- [8] J. Devlin. Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805, 2018. [1](#)
- [9] D. Du, Y. Zhang, S. Cao, J. Guo, T. Cao, X. Chu, and N. Xu. Bitdistiller: Unleashing the potential of sub-4-bit llms via self-distillation. arXiv preprint arXiv:2402.10631, 2024. [2](#)
- [10] E. Frantar, S. Ashkboos, T. Hoefer, and D. Alistarh. Optq: Accurate quantization for generative pre-trained transformers. In ICLR, 2022. [3](#)
- [11] X. Garcia, Y. Bansal, C. Cherry, G. Foster, M. Krikun, M. Johnson, and O. Firat. The unreasonable effectiveness of few-shot learning for machine translation. In ICML. PMLR. [1](#)
- [12] R. M. Gray and D. L. Neuhoff. Quantization. IEEE transactions on information theory, 44(6):2325–2383, 1998. [2](#)
- [13] S. Han, H. Mao, and W. J. Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. In ICLR, 2016. [1](#)
- [14] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. In NeurIPS Workshop, 2014. [1](#)
- [15] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami. Kvquant: Towards 10 million context length llm inference with kv cache quantization. arXiv preprint arXiv:2401.18079, 2024. [3](#)
- [16] H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. P. Graf. Pruning filters for efficient convnets. 2017. [5](#)
- [17] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. In MLSys, 2024. [1, 2, 3, 5, 7](#)
- [18] Z. Liu, B. Oguz, C. Zhao, E. Chang, P. Stock, Y. Mehdad, Y. Shi, R. Krishnamoorthi, and V. Chandra. Llm-qat: Data-free quantization aware training for large language models. arXiv preprint arXiv:2305.17888, 2023. [2](#)
- [19] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. arXiv preprint arXiv:2402.02750, 2024. [3](#)
- [20] X. Ma, G. Fang, and X. Wang. Llm-pruner: On the structural pruning of large language models. In NeurIPS, 2023. [1](#)
- [21] S. Merity, C. Xiong, J. Bradbury, and R. Socher. Pointer sentinel mixture models. arXiv preprint arXiv:1609.07843, 2016. [7](#)
- [22] T. Mihaylov, P. Clark, T. Khot, and A. Sabharwal. Can a suit of armor conduct electricity? a new dataset for open book question answering. arXiv preprint arXiv:1809.02789, 2018. [7](#)
- [23] G. Park, B. Park, M. Kim, S. Lee, J. Kim, B. Kwon, S. J. Kwon, B. Kim, Y. Lee, and D. Lee. Lut-gemm: Quantized matrix multiplication based on luts for effi-

- cient inference in large-scale generative language models. arXiv preprint arXiv:2206.09557, 2022. [3](#)
- [24] R. Saha, V. Srivastava, and M. Pilanci. Matrix compression via randomized low rank and low precision factorization. In NeurIPS, 2023. [2](#)
- [25] K. Sakaguchi, R. L. Bras, C. Bhagavatula, and Y. Choi. Winogrande: An adversarial winograd schema challenge at scale. Communications of the ACM, 64(9):99–106, 2021. [7](#)
- [26] M. Sap, H. Rashkin, D. Chen, R. LeBras, and Y. Choi. Socialqa: Commonsense reasoning about social interactions. arXiv preprint arXiv:1904.09728, 2019. [7](#)
- [27] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In NeurIPS, 2017. [1](#)
- [28] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han. Smoothquant: Accurate and efficient post-training quantization for large language models. In ICML, 2023. [3](#)
- [29] Z. Yao, R. Yazdani Aminabadi, M. Zhang, X. Wu, C. Li, and Y. He. Zeroquant: Efficient and affordable post-training quantization for large-scale transformers. In NeurIPS, 2022. [3](#)
- [30] Z. Yuan, L. Niu, J. Liu, W. Liu, X. Wang, Y. Shang, G. Sun, Q. Wu, J. Wu, and B. Wu. Rptq: Reorder-based post-training quantization for large language models. arXiv preprint arXiv:2304.01089, 2023. [3](#)
- [31] Y. Yue, Z. Yuan, H. Duanmu, S. Zhou, J. Wu, and L. Nie. Wkvquant: Quantizing weight and key/value cache for large language models gains more. arXiv preprint arXiv:2402.12065, 2024. [3](#)
- [32] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi. Hellaswag: Can a machine really finish your sentence? arXiv preprint arXiv:1905.07830, 2019. [7](#)
- [33] Y. Zhang. Dialogpt: Large-scale generative pre-training for conversational response generation. arXiv preprint arXiv:1911.00536, 2019. [1](#)
- [34] W. Zhu, H. Liu, Q. Dong, J. Xu, S. Huang, L. Kong, J. Chen, and L. Li. Multilingual machine translation with large language models: Empirical results and analysis. arXiv preprint arXiv:2304.04675, 2023. [1](#)